Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
DEEMED TO BE
University
(Estd. u/s 3 of UGC Act, 1956)
Avadi, Chennai

School of Computing

Department of Computer Science and Engineering USE

CASE REAL LIFE EXAMPLE

Program- Department Name	**B.Tech – CSE**
Academic Year/Semester	2026-27 / Summer
Faculty Name	Dr. S. Hemamalini
Course Code/Course Name	10211CS227 / Problem solving and Testing
Slot No & Date	S3-2 / 22-10-2026
Student Name	D Revanth Reddy
Title of the Complex Engineering problem / problems	SUPER MARKET BILLING SYSTEM
Description / Complexity Addressed	The Super Market Billing System addresses the complexities involved in managing supermarket products, processing customer bills, calculating discounts, and maintaining accurate inventory records. The system allows users to add and remove products, calculate the total bill based on quantity and price, apply discounts, update stock after sales, and generate sales reports. It validates user inputs, prevents billing for unavailable products or insufficient stock, and handles invalid transactions. It demonstrates the use of programming constructs, methods, conditional statements.
CO's Mapped	CO1, CO2, CO3, CO4, CO5
PO's / PSO's Mapped	PO1, PO2, PO3, PO5, PSO2
SDG Mapped with number	SDG 12: Responsible Consumption and Production.
Justification for SDG Mapped	The Super Market Billing System supports SDG 12 – Responsible Consumption and Production by promoting efficient inventory management and reducing product wastage. The system maintains accurate stock records, updates product quantities after sales, and helps identify low-stock products. Automated billing reduces manual calculation errors and improves resource utilization. Thus, the system encourages responsible stock management and supports sustainable supermarket operations.
Proof for the solved Complex Engineering Problem	https://drive.google.com/drive/folders/1xRIlDrZpvEvbM77e1Vs3WojLz105Gwje?usp=drive_link

Signature of the Faculty

TABLE OF CONTENTS

S. No.	Contents	Page No.
1.	Use-case	4-5
2.	Problem Statement	6
3.	Code	7-12
4.	Output Screenshot	13-14
5.	Result	15

1.CASE STUDY:

The Super Market Billing System is a software application designed to simplify and automate the billing and inventory management activities of a supermarket. The system allows the administrator to add and remove products, maintain product details, generate customer bills, calculate discounts, and display inventory information. The main objective of this system is to reduce manual calculation errors, save time during billing, and maintain accurate records of products and sales.

The system demonstrates important programming concepts such as:

- Product Management: Adding, updating, and removing products from inventory.
- Billing Operations: Calculating the total price based on product quantity and unit price.
- Discount Calculation: Applying discounts to the total bill when applicable.
- Inventory Management: Tracking available stock and updating quantities after sales.
- Validation and Error Handling: Handling invalid quantities, unavailable products, and incorrect menu choices.
- Sales Analysis: Identifying the most sold products and displaying sales information.

Purpose of the System

The purpose of the Super Market Billing System is to provide a simple, reliable, and efficient way to manage supermarket billing and stock information. It improves billing accuracy, reduces paperwork, and helps supermarket staff manage daily sales and inventory effectively.

Block Diagram:

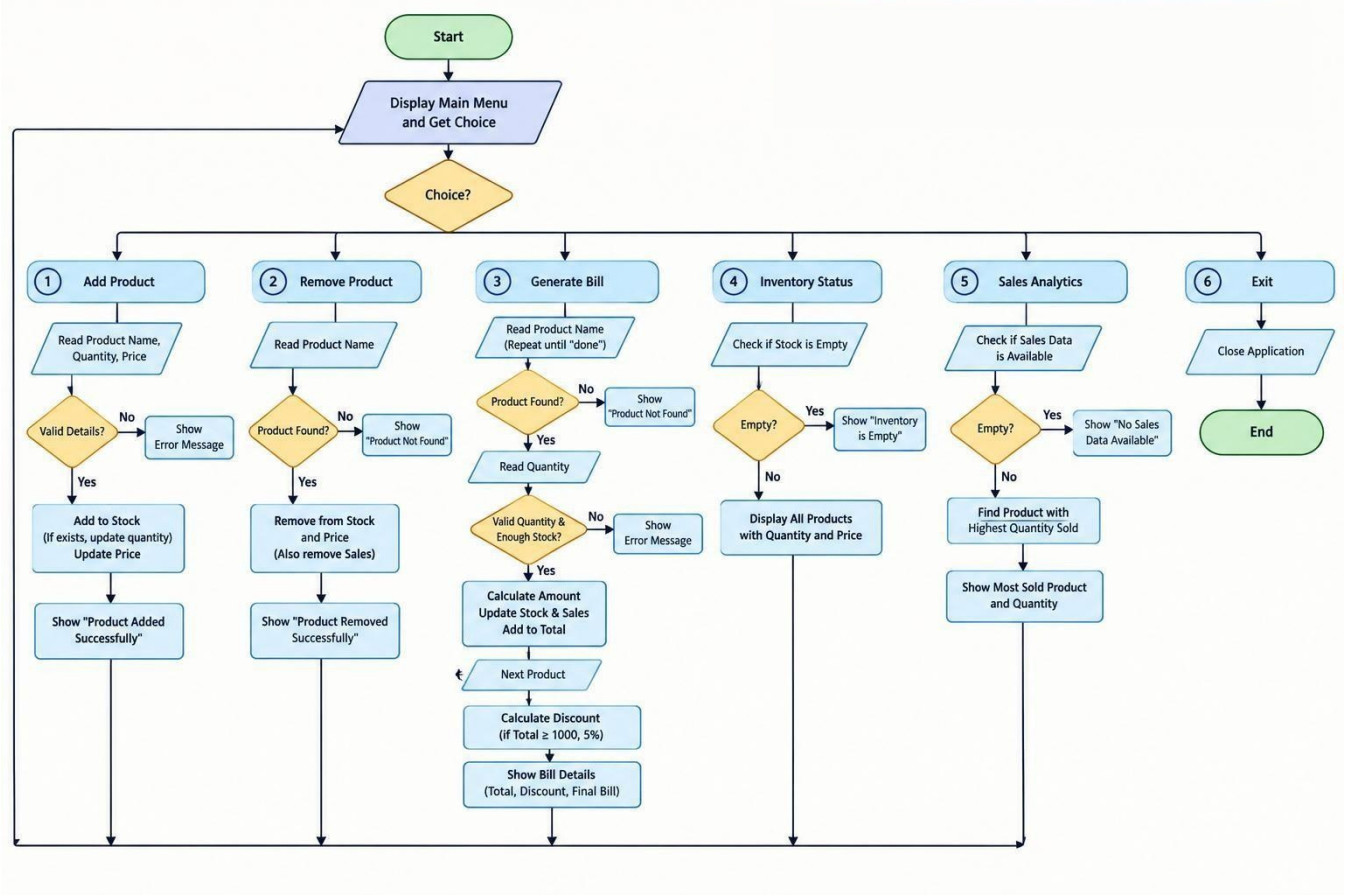


Fig:1 Supermarket Billing System

2.Problem Statement:

The Super Market Billing System aims to solve the difficulties involved in manual billing and inventory management in supermarkets. Manual billing requires employees to calculate product prices, quantities, discounts, and total amounts, which may lead to calculation errors, delays, and inaccurate stock records.

The proposed system automates these operations by providing a menu-driven application that manages products, generates bills, validates user inputs, and maintains inventory details.

Functional Requirements:

- Product Addition: Add new products with their name, quantity, and price.
- Product Removal: Remove products from the inventory when required.
- Bill Generation: Calculate the total amount for purchased products and generate an invoice.
- Discount Application: Deduct the applicable discount from the total bill.
- Inventory Status: Display available products and their remaining quantities.
- Sales Analytics: Display the most sold product and peak sales information when sales records are available.
- Input Validation: Reject negative or zero quantities where inappropriate, invalid prices, and unavailable products.
- Menu Validation: Display an error message when the user selects an invalid menu option.
- Exit: Terminate the application safely.

Complexity Addressed

The system addresses problems involving product data management, repeated billing calculations, stock updates, discount processing, and input validation. It demonstrates the use of arrays or collections, methods, conditional statements, loops, exception handling, and basic data-processing techniques.

Expected Outcome

The system should generate accurate bills, maintain updated inventory records, reject invalid transactions, and provide useful sales information. This improves the speed, accuracy, and reliability of supermarket operations.

1. Code

```
import java.util.*;


public class Main {


    static Scanner sc = new Scanner(System.in);
```

```
   static Map<String, Integer> stock = new HashMap<>();
static Map<String, Double> price = new HashMap<>();     static
Map<String, Integer> sales = new HashMap<>();

   public static void main(String[] args) {

       while (true) {

           System.out.println("\n===== SUPERMARKET BILLING SYSTEM
=====");
           System.out.println("1. Add Product");
           System.out.println("2. Remove Product");
           System.out.println("3. Generate Bill");
           System.out.println("4. Inventory Status");
           System.out.println("5. Sales Analytics");
           System.out.println("6. Exit");
           System.out.print("Enter Choice: ");
            int
choice;
           try {               choice
= sc.nextInt();
           } catch (InputMismatchException e) {
System.out.println("Invalid Choice");
sc.nextLine();               continue;
           }

           sc.nextLine();

           switch (choice) {

              // 1. Add Product
case 1:
                   System.out.print("Enter Product Name: ");
                   String name = sc.nextLine();
```

```
                System.out.print("Enter Quantity: ");
int qty = sc.nextInt();

                System.out.print("Enter Price: ");
double p = sc.nextDouble();
sc.nextLine();

                if (name.trim().isEmpty() || qty <= 0 || p <= 0) {
                   System.out.println("Invalid Product Details");
                } else {                    stock.put(name,
stock.getOrDefault(name, 0) + qty);                    price.put(name,
p);

                   System.out.println("Product Added Successfully");
                }
break;

            // 2. Remove Product
case 2:
                System.out.print("Enter Product Name: ");
name = sc.nextLine();

                if (stock.remove(name) != null) {
price.remove(name);                    sales.remove(name);

                   System.out.println("Product Removed Successfully");
                } else {
                   System.out.println("Product Not Found");
                }
break;

            // 3. Generate Bill
case 3:
                double total = 0;

                while (true) {
```

```
                System.out.print("Enter Product Name (done to finish): ");
name = sc.nextLine();

                if (name.equalsIgnoreCase("done")) {
break;
                }

                if (!stock.containsKey(name)) {
                    System.out.println("Product Not Found");
continue;
                }

                System.out.print("Enter Quantity: ");
                 try {
qty = sc.nextInt();
                } catch (InputMismatchException e) {
                    System.out.println("Invalid Quantity");
                    sc.nextLine();
continue;
                }

sc.nextLine();

                if (qty <= 0 || qty > stock.get(name)) {
                    System.out.println("Invalid Quantity or Insufficient Stock");
continue;
                }

                double amount = qty * price.get(name);
total += amount;

                stock.put(name, stock.get(name) - qty);
sales.put(name, sales.getOrDefault(name, 0) + qty);

                System.out.printf("%s: Rs. %.2f%n", name, amount);
            }
```

```
                if (total > 0) {                    double discount =
total >= 1000 ? total * 0.05 : 0;

                    System.out.printf("Total: Rs. %.2f%n", total);
                    System.out.printf("Discount: Rs. %.2f%n", discount);
                    System.out.printf("Final Bill: Rs. %.2f%n", total - discount);
} else {
                    System.out.println("No Items Purchased");
                }
break;

            // 4. Inventory Status
case 4:
                if (stock.isEmpty()) {
                    System.out.println("Inventory is Empty");
                } else {
                    System.out.println("\n----- INVENTORY STATUS -----");

                    for (String item : stock.keySet()) {
System.out.println(                         item + " -
Quantity: " + stock.get(item)
                            + ", Price: Rs. " + price.get(item)
                        );
                    }
}
break;

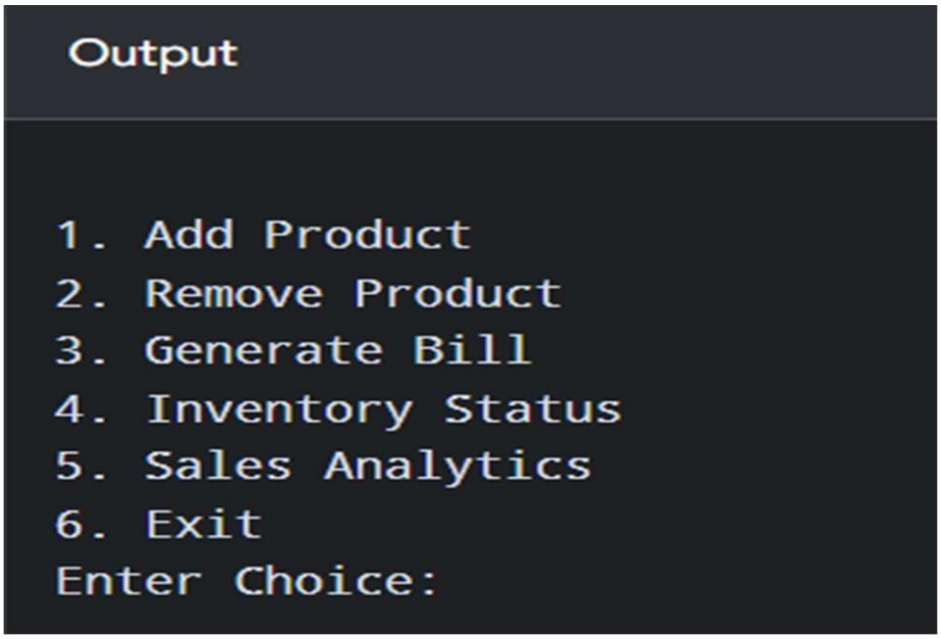
            // 5. Sales Analytics
case 5:
                if (sales.isEmpty()) {
                    System.out.println("No Sales Data Available");
                } else {
                    String best = Collections.max(
sales.entrySet(),
                        Map.Entry.comparingByValue()
                    ).getKey();
```

```
                System.out.println("Most Sold Product: " + best);
                System.out.println("Quantity Sold: " + sales.get(best));
            }
break;

            // 6. Exit
case 6:
            System.out.println("Application Closed");
sc.close();
            return;

default:
            System.out.println("Invalid Choice. Enter 1 to 6");
        }
    }
  }
}
```

2. OUTPUT SCREENSHOT



Output

```
1. Add Product
2. Remove Product
3. Generate Bill
4. Inventory Status
5. Sales Analytics
6. Exit
Enter Choice: 1
Enter Product Name: RICE
Enter Quantity: 10
Enter Price: 500
Product Added Successfully
```

```
1. Add Product
2. Remove Product
3. Generate Bill
4. Inventory Status
5. Sales Analytics
6. Exit
Enter Choice: 3
Enter Product Name (done to finish): RICE
Enter Quantity: 2
RICE: 1000.00
Enter Product Name (done to finish): DONE
Total: Rs. 1000.00
Discount: Rs. 50.00
Final Bill: Rs. 950.00

1. Add Product
2. Remove Product
3. Generate Bill
4. Inventory Status
5. Sales Analytics
6. Exit
Enter Choice:

=== Session Ended Due To Inactivity ===
```

5.Result :

The Super Market Billing System was designed to automate product management, billing, discount calculation, inventory updates, and sales analysis. The system validates user inputs, handles invalid product requests, and calculates the final bill accurately. Testing with valid and invalid inputs helps verify the correctness and reliability of the application. Thus, the objective of developing a simple and efficient Super Market Billing System using Java programming concepts was achieved.

Table I: Test Cases and Execution Results for the Super Market Billing System

Test Case ID	Choice	Test Scenario	Input / Action	Expected Output	Execution Output (Copy the execution sequence of this choice)	Status
TC01	1	Valid Product Addition	Product: Rice, Quantity: 10, Price: ₹500	Product should be added successfully to inventory	Enter Choice: 1 Enter Product Name: Rice Enter Quantity: 10 Enter Price: ₹500 Product Added Successfully	Pass
TC02	1	Invalid Product Addition	Product: Rice, Quantity: -2, Price: ₹500	System should reject invalid quantity	Enter Choice: 1 Enter Product Name: Rice Enter Quantity: -2 Invalid Quantity. Please enter a valid quantity.	Pass
TC03	2	Valid Product Removal	Product: Sugar	Product should be removed successfully from inventory	Enter Choice: 2 Enter Product Name: Sugar Product Removed Successfully	Pass
TC04	2	Invalid Product Removal	Product: XYZ	System should display product not found message	Enter Choice: 2 Enter Product Name: XYZ Product Not Found	Pass

TC05	3	Generate Bill	Rice: Qty 2, Price ₹500; Sugar: Qty 1, Price ₹60; Discount ₹50	Total bill should be calculated and invoice generated	Enter Choice: 3 Enter Product Name: Rice Enter Quantity: 2 Rice: 1000.00 Enter Product Name: Sugar Enter Quantity: 1 Sugar: 60.00 Enter Product Name: done Total: Rs. 1060.00	Pass
					Discount: Rs. 53.00 Final Bill: Rs. 1007.00	
TC06	3	Invalid Billing Quantity	Product: Rice, Quantity: 0	System should reject zero quantity	Enter Choice: 3 Enter Product Name: Rice Enter Quantity: 0 Invalid Quantity or Insufficient Stock	Pass
TC07	4	Inventory Status	Select Choice 4	Current inventory details should be displayed	Enter Choice: 4 Rice - Quantity: 8, Price: Rs. 500.0	Pass
TC08	4	Empty Inventory Status	Select Choice 4 when no products are available	System should indicate that inventory is empty	Enter Choice: 4 Inventory is Empty	Pass

TC09	5	Sales Analytics	Select Choice 5	Most sold product and peak sales time should be displayed	Enter Choice: 5 Most Sold Product: Rice Quantity Sold: 2	Pass
TC10	5	No Sales Data	Select Choice 5 before any sales	System should display that no sales data is available	Enter Choice: 5 No Sales Data Available	Pass
TC11	6	Exit Application	Select Choice 6	Application should terminate successfully	Enter Choice: 6 Application Closed	Pass
TC12	Invalid	Invalid Menu Choice	Enter Choice: 8	System should reject invalid menu option	Enter Choice: 8 Invalid Choice. Enter 1 to 6	Pass